

Lecture 5: Lasso, Cross-validation

Max G'Sell
Machine Learning I: 46-926

November 11, 2020

Announcements

- ▶ Homework 2 was turned in, let's talk about it.

Today: Continuing the Lasso, Cross-validation

- ▶ Finishing our discussion of the Lasso.
- ▶ Model selection, cross-validation

Review: Supervised learning

Observe $(X_1, Y_1), \dots, (X_n, Y_n) \stackrel{iid}{\sim} \mathcal{P}$. Estimate $\hat{\mu}(x)$ so that $\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X))$ is small for new $(X, Y) \sim \mathcal{P}$.

If we are minimizing expected squared prediction error:

- If we actually knew $\mathcal{P}$, the best we could do is

$$\mu(x) = \mathbb{E}(Y|X = x)$$

- If we are estimating $\hat{\mu}$, then

$$\mathbb{E}(\mathcal{L}(Y, \hat{\mu}(X)) \mid X = x) = \sigma_x^2 + \underbrace{\text{bias}(\hat{\mu}(x))^2}_{\text{bias}^2} + \underbrace{\text{var}(\hat{\mu}(x))}_{\text{variance}}$$

Review: Linear regression

We can think of linear regression as one way to approximate $\mathbb{E}(Y|X = x)$ across x values, imagining:

$$\mathbb{E}(Y|X = x) \approx x^T \beta$$

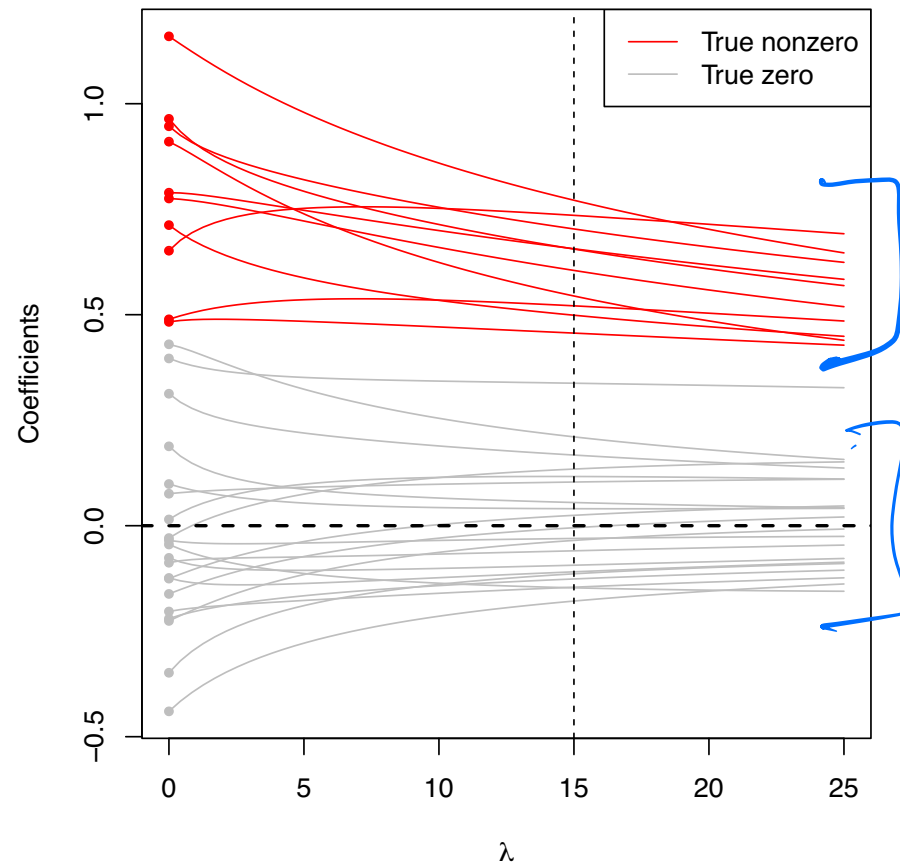
This can work well if the linear model is not a bad approximation.

However, we noticed that, as the number of variables, p , grows large, the variance of linear regression grows rapidly. This leads to poor predictions **even with the linear model is true!**

Ridge regression coefficients

We saw that ridge regression reduced variance, but in an unappealing way: large coefficients shrink a lot, small coefficients only shrink a little.

Ridge regression coefficients, plotted against λ :



We introduced the lasso to address this.

The lasso

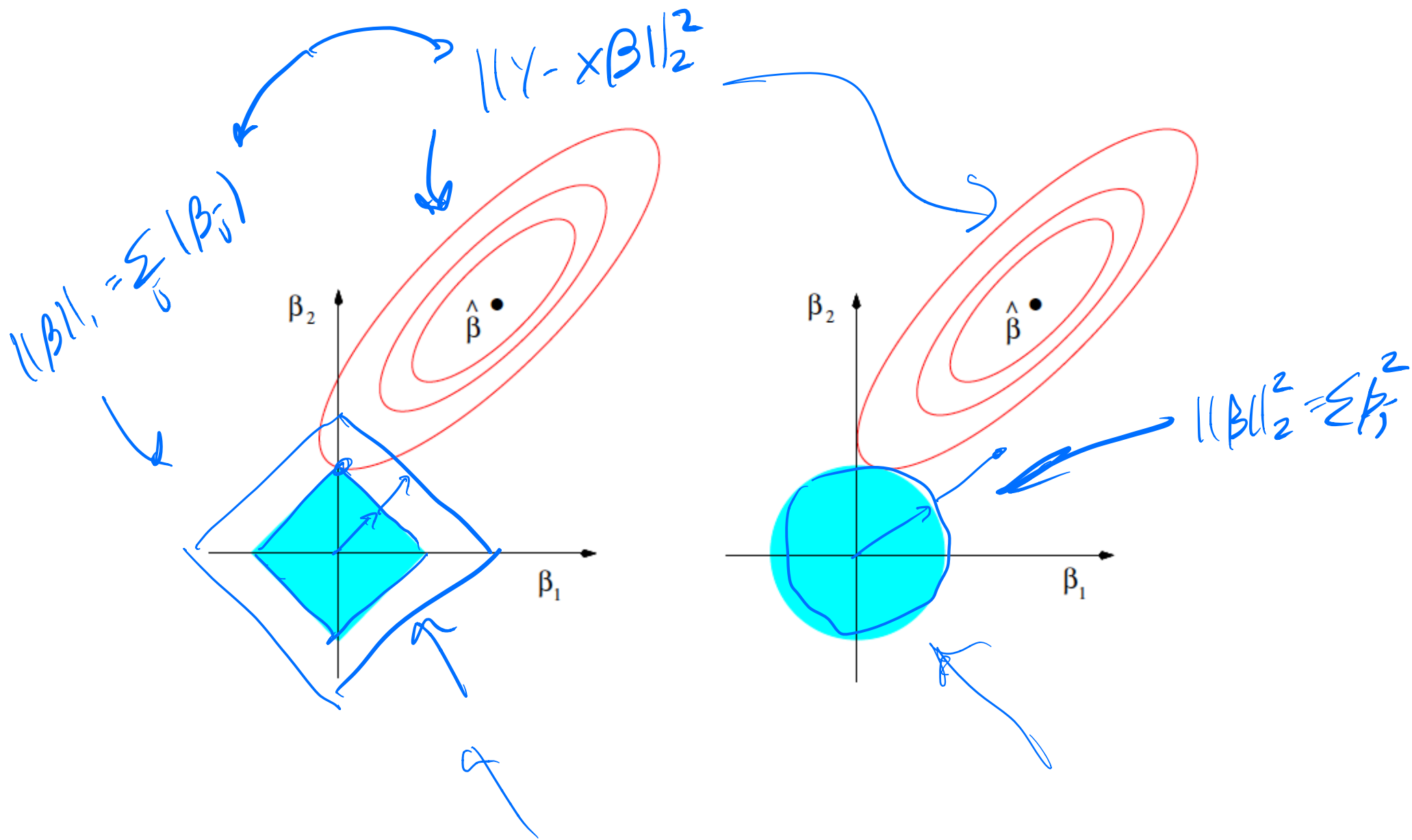
The **lasso**¹ estimate is defined as

$$\begin{aligned}\hat{\beta}^{\text{lasso}} &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^p |\beta_j| \\ &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \underbrace{\lambda \|\beta\|_1}_{\text{Penalty}}\end{aligned}$$

The squared ℓ_2 penalty $\|\beta\|_2^2$ of ridge regression, has been replaced by an ℓ_1 penalty $\|\beta\|_1$. Even though these problems look similar, their solutions behave very differently

Note the name “lasso” is actually an acronym for: Least Absolute Selection and Shrinkage Operator

¹Tibshirani (1996), “Regression Shrinkage and Selection via the Lasso”



$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

The **tuning parameter** λ controls the strength of the penalty.

- ▶ If $\lambda = 0$:
- ▶ As $\lambda \rightarrow \infty$:

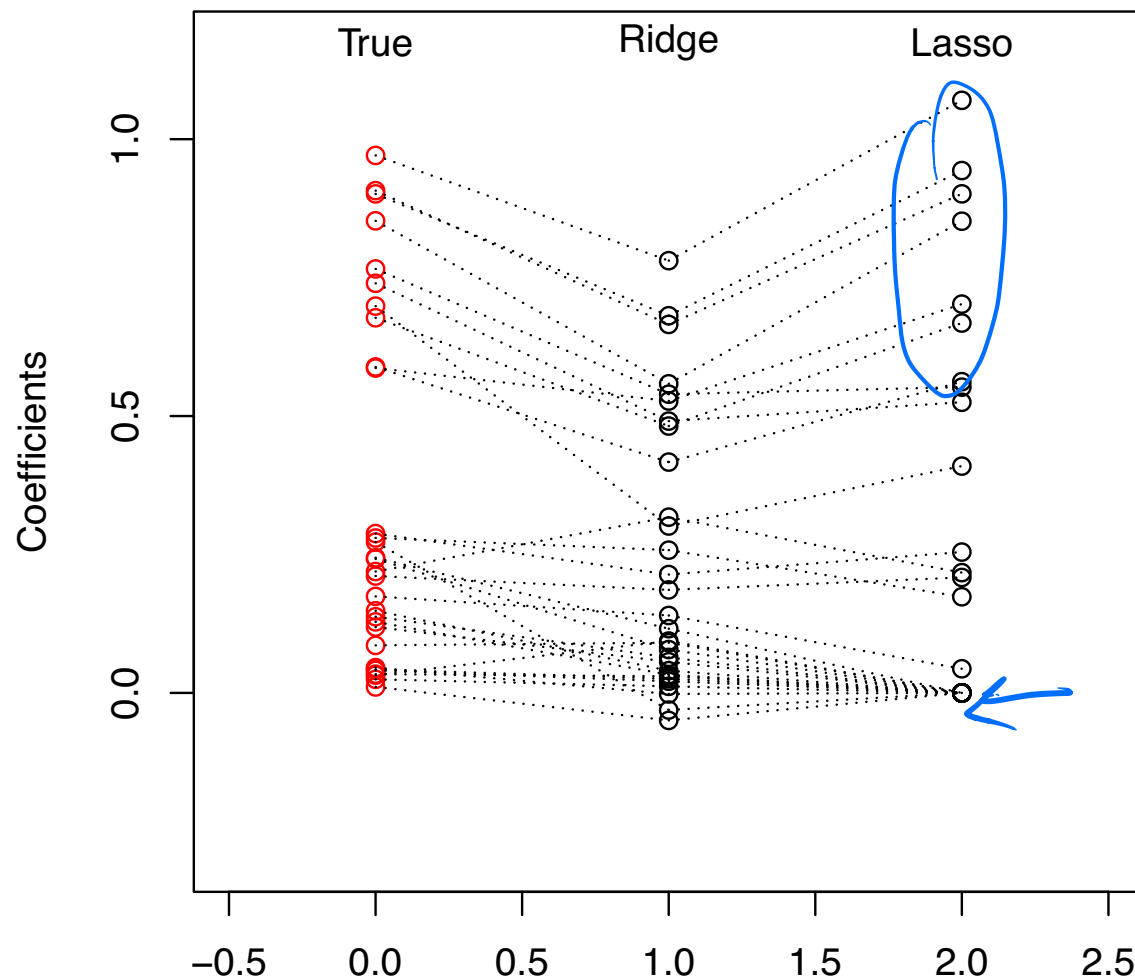
$\lambda_{\max}$

For λ in between these two extremes, we are balancing two ideas: fitting a linear model of y on X , and shrinking the coefficients. But the nature of the ℓ_1 penalty causes some coefficients to be shrunk to **zero exactly**

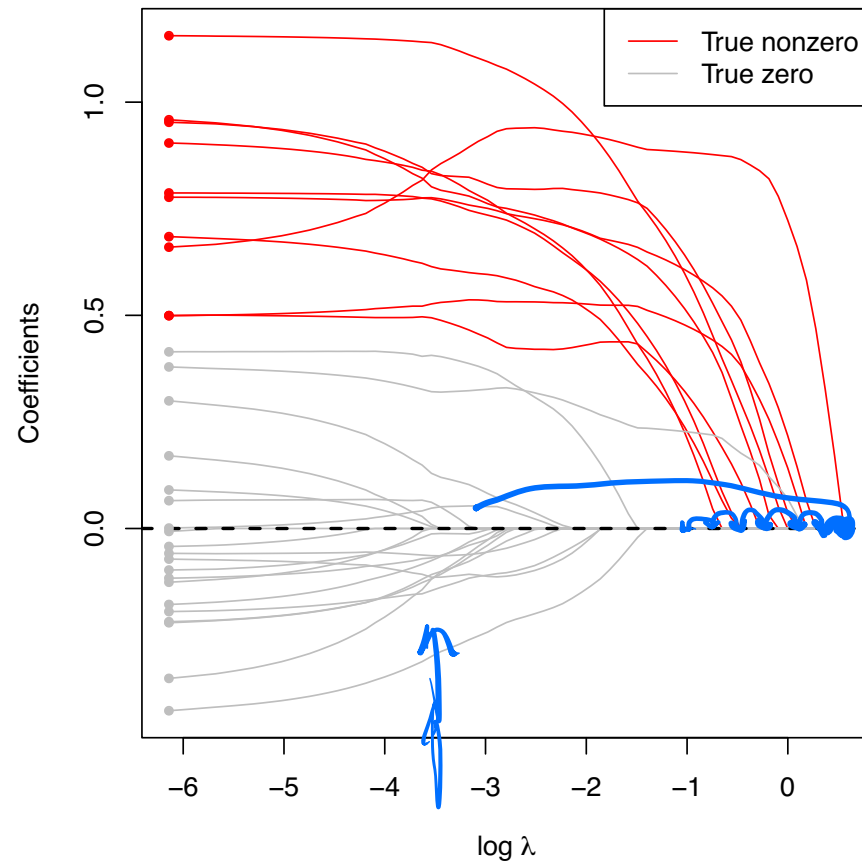
This leads to **variable selection**!

Example: visual representation of lasso coefficients

Our running example from last time: $n = 50$, $p = 30$, $\sigma^2 = 1$, 10 large true coefficients, 20 small. Here is a visual representation of lasso vs. ridge coefficients (with the same degrees of freedom):



Lasso coefficient paths



In the lasso, we get exact zeros! Moreover, the large coefficients are not impacted nearly as much!

It is often easier to view lasso results on the $\log \lambda$ scale.

Advantages of sparsity

- ▶ Interpretability: We can understand what the model relies on for prediction (understanding $\hat{f}$)
- ▶ We might gain some insight into the underlying data (though not causally) (helping to understand f)
- ▶ If we're building a predictive score, we can measure fewer things in the future (simpler $\hat{f}$ to apply later)

Sidenote: Can we do inference (e.g., p -values, confidence intervals, etc)? **Not easily!**

Important details

Just like in ridge regression:

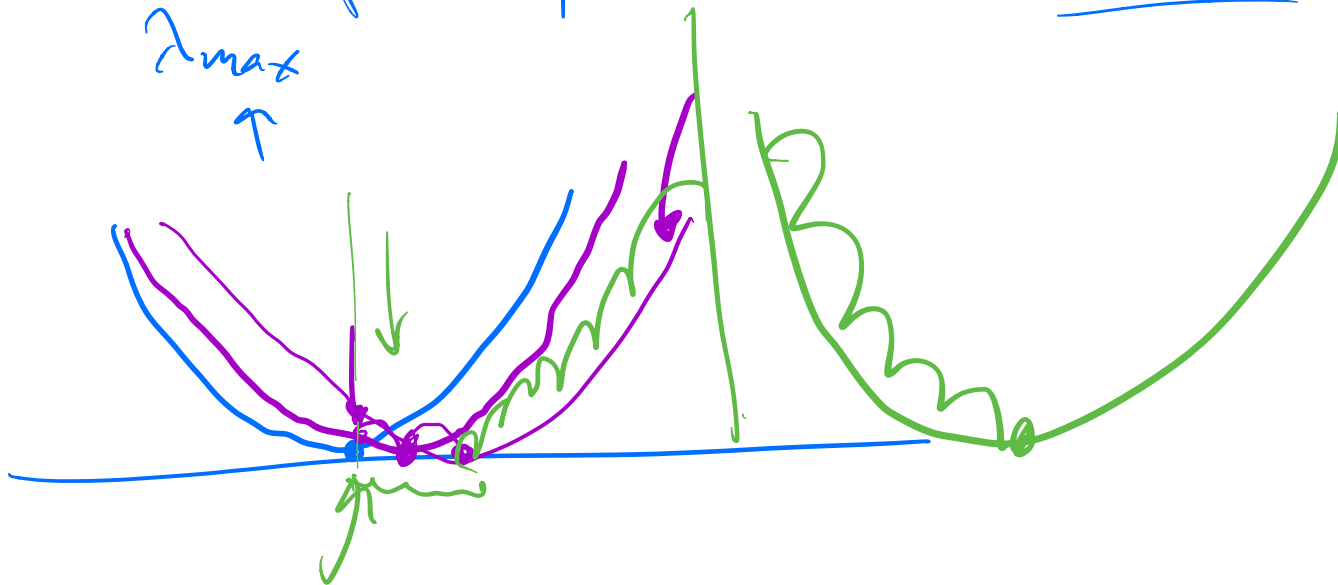
- ▶ We don't penalize our intercept, if one is included. Again, columns are often centered and then intercept omitted.
- ▶ The penalty term $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$ is not fair if the predictor variables are **not on the same scale**. Hence, we often scale the columns of X to have variance 1.

How is it actually solved?

glmnet

$$\min_{\beta} \underbrace{\|Y - X\beta\|_2^2}_{\text{RSS}} + \underbrace{2\lambda \|\beta\|_1}_{\text{penalty}}$$

Warm Starts



[REMINDER: SHOW SOFTWARE DEMOS]

Interview question

$$Y = X_1 + X_2$$

Suppose that you regress Y on X_1, X_2 using both ridge regression and the lasso (with $\lambda > 0$). If $X_1 = X_2$, what does: $\beta_1 + \beta_2 = c$ ←

The solution to ridge look like

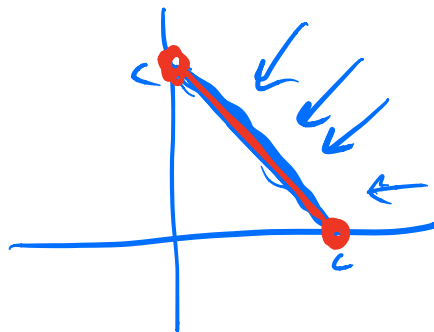
$$\beta_1 = \beta_2$$

$$\|Y - X_1\beta_1 - X_2\beta_2\|_2^2 + \lambda\beta_1^2 + \lambda\beta_2^2$$

$\lambda(\beta_1^2 + \beta_2^2)$

The solution to lasso look like

$$\|Y - X_1\beta_1 - X_2\beta_2\|_2^2 + \lambda(|\beta_1| + |\beta_2|)$$



pick β_1 or $\beta_2 = 0$.

Simple case: Orthogonal X

Consider the very simple case where $X = I$. We can examine how both ridge regression and the lasso behave on this data. This is easy, because the variables do not interact with one another, but it gives an intuition for their more general behavior.

When $X = I$, note that the RSS becomes

$$\|y - X\beta\|_2^2 = \|y - \beta\|_2^2 = \sum_{i=1}^n (y_i - \beta_i)^2.$$

Simple case: Orthogonal X

The corresponding penalized forms then become

Ridge:

$$\hat{\beta}_{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_i)^2 + \lambda \beta_i^2$$

Lasso:

$$\hat{\beta}_{\text{lasso}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_i)^2 + \lambda |\beta_i|$$

We can minimize separately for each i !

For ridge regression, differentiating yields

and therefore

$$\hat{\beta}_i =$$

Thus the estimates from ridge regression correspond to the least squares estimates reduced by a constant multiple.

Note that this can never give zero coefficients, and that it penalizes large coefficients quite a bit.

For the lasso, it turns out that the minimizer of

$$(y_i - \beta_i)^2 + \lambda|\beta_i|$$

is given by

$$\hat{\beta}_i =$$

Thus estimates from ridge regression correspond to shrinking the least squares estimates toward zero by an additive constant (without crossing zero).

This can give zero coefficients, and is also relatively harsher on small coefficients than larger ones. It appears to be better suited to sparse settings.

Regularized regression

Least squares solves the problem:

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2$$

In regularized regression, we add a penalty to this squared error loss term

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2 + P_{\lambda}(\beta)$$

You've seen several examples: $P_{\lambda}(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_{\lambda}(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both “pull” β toward 0, causing it to be less variable. However, by making β smaller than the least squares estimate, they add bias.

The λ parameter determines how strong this penalty is. A larger λ leads to a higher bias and a smaller variance.

Regularized regression

$$\operatorname{argmin}_{\beta} \|y - X\beta\|_2^2 + P_{\lambda}(\beta)$$

$P_{\lambda}(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_{\lambda}(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both “pull” β toward 0, causing it to be less

However, by making β smaller than the least squares estimate, they add

The λ parameter determines how strong this penalty is. A larger λ leads to a higher bias and a smaller variance

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

We will see at least:

- ▶ Logistic regression with a lasso penalty
- ▶ The elastic net

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

Other classic examples:

- ▶ The group lasso: setting groups of variables to zero
- ▶ Graphical lasso: estimates networks of dependence between variables
- ▶ Fused lasso: estimates piecewise-constant functions
- ▶ Trend filtering: estimates piecewise-polynomial functions
- ▶ Low-rank matrix decomposition: estimate a low-rank approximation to a matrix
- ▶ Sparse additive models
- ▶ Vector auto-regressive models

Model selection and generalization error

Regularized regression is one of the first settings you see where you must pick a particular model by setting a tuning parameter (λ here).

We know that λ trades off between bias and variance, and that we want to pick this tradeoff to minimize the *generalization error* for *new* samples.

How do we actually pick this λ ?

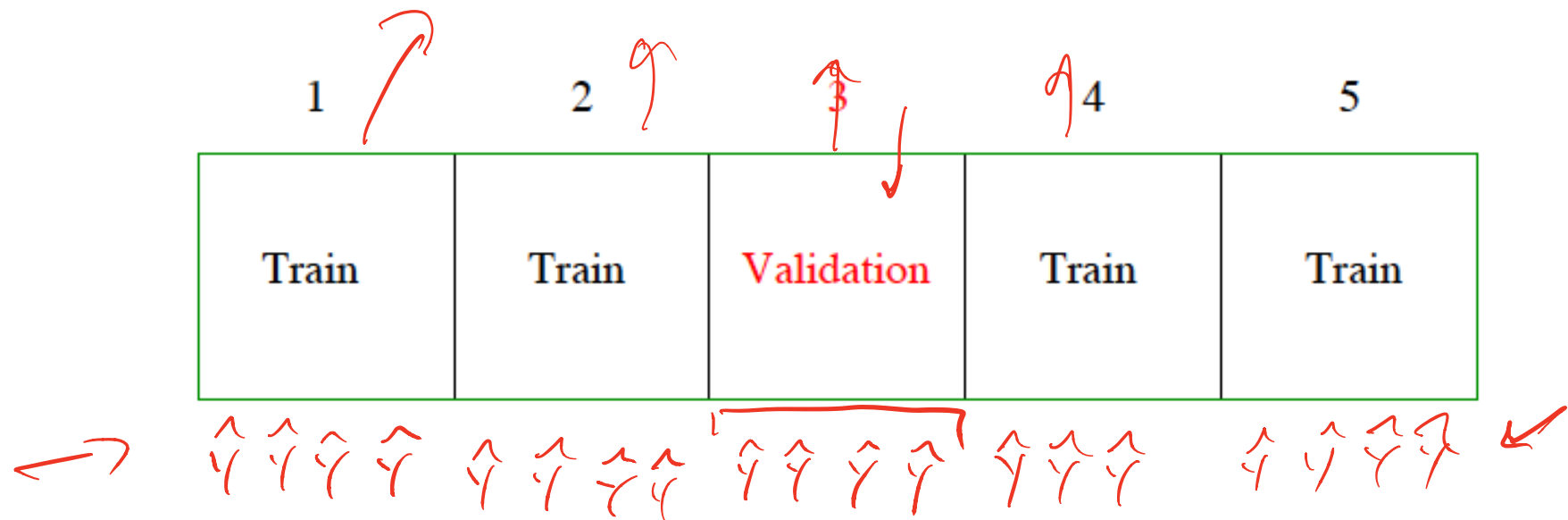
[WARNING: We are going to talk about this for now in the i.i.d. setting. We need to be more careful for timeseries. We will come back to this.]

K -fold Cross-validation

We want to estimate how well a method with each λ should do on *new* data points.

We don't have new data points, but we can hold out a few of our own.

We split the training pairs into K parts or “folds” (commonly $K = 5$ or $K = 10$)



K -fold cross validation considers training on all but the k th part, and then validating on the k th part, iterating over $k = 1, \dots, K$

K -fold Cross-validation

We want to estimate how well a method with each λ should do on *new* data points.

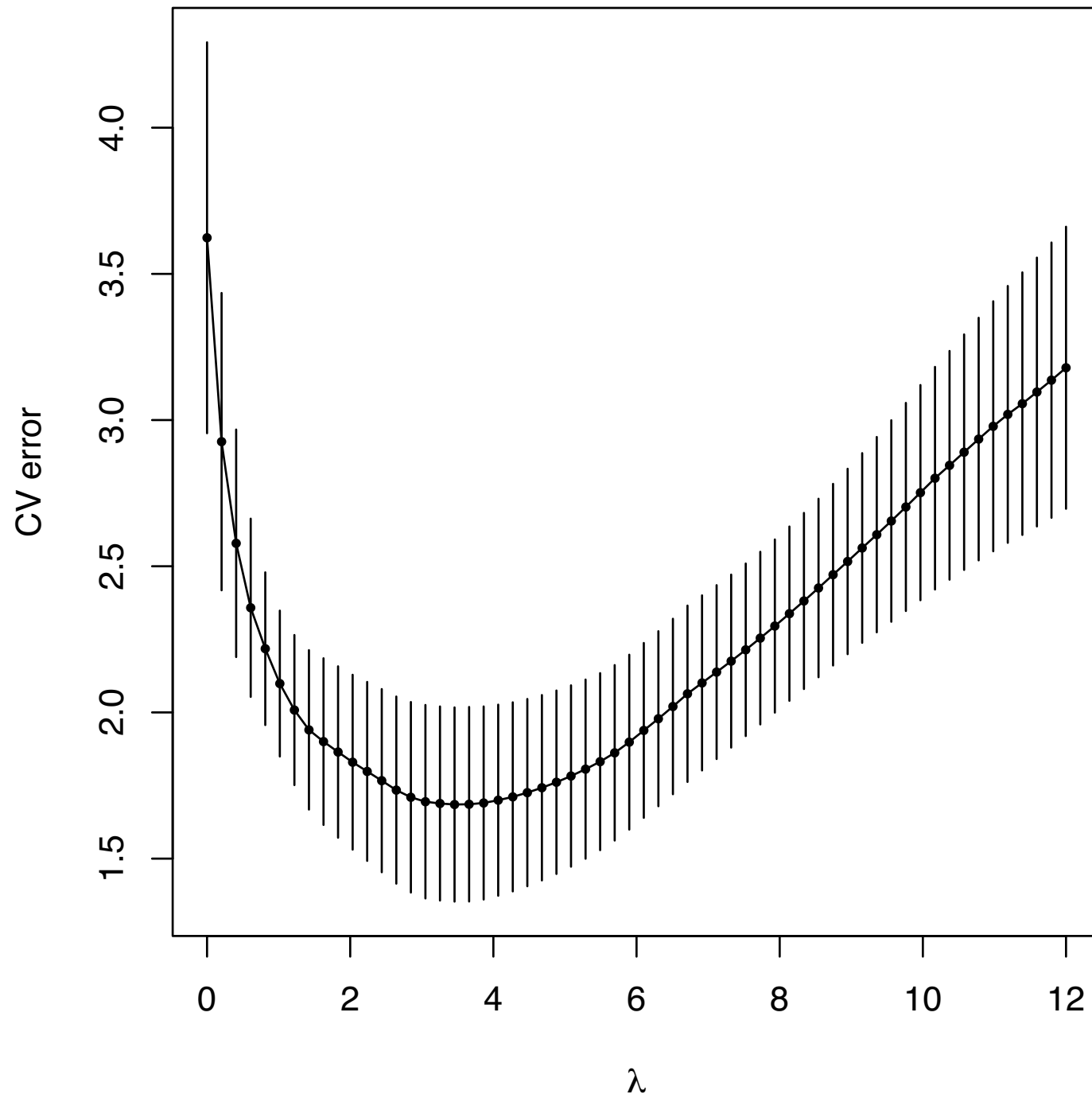
We don't have new data points, but we can hold out a few of our own.

K -fold Cross Validation:

1. Randomly split the data into K equal-sized parts (“folds”)
2. Each part is used as the test set once, with the other $K - 1$ parts combined into a training set
3. The error on all these test sets is averaged

These errors are used to select a final model (λ value) to use. The model is refit on the entire data set for this λ value.

Usually $K = 5, 10$, or n .



Standard errors for cross-validation

One nice thing about K -fold cross-validation (for a small $K \ll n$, e.g., $K = 5$) is that we can estimate the standard deviation of $CV(\theta)$, at each $\theta \in \{\theta_1, \dots, \theta_m\}$

First, we just average the validation errors in each fold:

$$CV_k(\theta) = \frac{1}{n_k} e_k(\theta) = \frac{1}{n_k} \sum_{i \in F_k} (y_i - \hat{f}_{\theta}^{-k}(x_i))^2$$

where n_k is the number of points in the k th fold

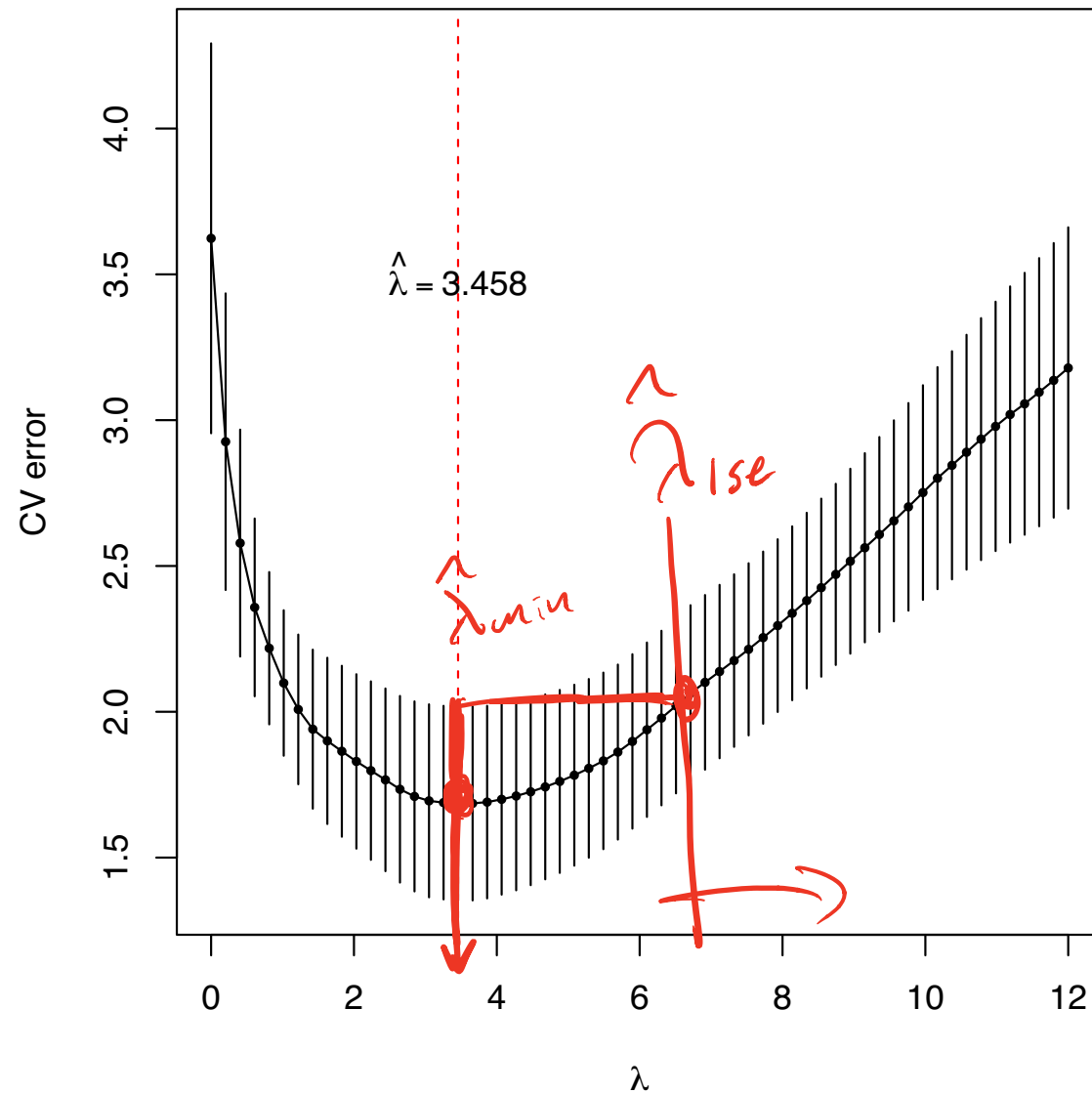
We then compute the sample standard deviation of $CV_1(\theta), \dots, CV_K(\theta)$,

$$SD(\theta) = \sqrt{\text{var}(CV_1(\theta), \dots, CV_K(\theta))}$$

Finally we estimate the standard deviation of $CV(\theta)$ —called the **standard error** of $CV(\theta)$ —by $SE(\theta) = SD(\theta)/\sqrt{K}$

Example: choosing λ for the lasso (continued)

The cross-validation error curve from our lasso example, with $\pm$ standard errors:



The one standard error rule

The **one standard error rule** is an alternative rule for choosing the value of the tuning parameter, as opposed to minimizing the cross-validation error.

We find the usual minimizer $\hat{\theta}$, and then move θ in the direction of **increasing regularization** as much as we can, while keeping the CV error within one S.E. of $CV(\hat{\theta})$:

$$CV(\theta) \leq CV(\hat{\theta}) + SE(\hat{\theta})$$

Idea: “All else equal (up to one standard error), go for the simpler (more regularized) model”

Example: choosing λ for the lasso (continued)

Back to our lasso example: the one standard error rule chooses a larger value of λ (more regularization):

